



Google Developer Groups

On Campus • SRM Institute Of Science And Technology

GDG on Campus, SRM Recruitments 2026-27

Technical Domain

Welcome to the **Google Developer Groups on Campus SRM's Recruitments for the Technical Domain**.

Candidates are required to complete **at least one task from any one domain by 4th October 2026**.

Each task must be stored in a **public GitHub repository**.

Note: Add screenshots of your application in the GitHub repository.

Plagiarism Warning:

Candidates are reminded that **plagiarism will not be tolerated**. Any instance of plagiarized code will result in strict action, including the cancellation of their candidature.

For any queries regarding the tasks, don't hesitate to reach out to us at:

technical@gdgsrm.com

Frontend

1. QR Code Generator & Designer

Description:

Develop and deploy a web application that allows users to generate and customize QR codes. Users should be able to enter different types of information, customize the appearance of the QR code, preview the result in real time, and download the generated QR code.

The application should work entirely in the browser without requiring a backend.

Technologies:

- Choose a frontend framework (React, Vue.js, or Angular)
- HTML
- CSS
- JavaScript or TypeScript
- QR code library of choice
- Vercel/Netlify (for deployment)

Tasks:

1. QR Code Generation:

- Allow users to enter a URL or text.
- Generate the QR code in real time.
- Display a preview of the generated QR code.

2. Different QR Types:

- URL
- Plain Text
- Email
- Phone Number
- Wi-Fi
- Display appropriate inputs based on the selected type.

3. QR Customization:

- QR code size

- Foreground and background
- Error correction level
- Margin/padding
- Changes should update the preview immediately.

4. Presets:

- Provide predefined visual presets.
- Users should be able to modify settings after selecting a preset.

5. Download:

- Allow users to download the generated QR code as PNG.
- The downloaded QR code should match the preview.

6. Validation:

- Validate user input.
- Display suitable errors for invalid or incomplete information.

7. Scan Reliability:

- Ensure customized QR codes remain scannable.
- Warn users about customization choices that may affect readability.

8. Recent QR Codes:

- Store recently generated QR codes locally.
- Allow users to reuse them.
- Recent QR codes should remain after page refresh.

9. Responsive Design:

- Ensure the application works properly on desktop and mobile screens.

10. Testing:

- Test all supported QR types.
- Test customization options.
- Verify downloaded QR codes.
- Test invalid inputs.

- Verify local persistence.
- Check responsiveness.

Optional Enhancements:

- SVG download
- Add a logo to QR codes
- Gradient QR codes
- Copy to clipboard
- Custom QR patterns
- Dark/light theme

2. Interactive Product Showcase using Framer

Description:

Design and build an interactive product showcase using Framer. The showcase should present a fictional or existing digital product, application, or device through a visually appealing layout with smooth animations and interactions.

The task should be built as a **single section or experience**, not a complete website. The focus is on assessing Framer fundamentals, visual design, responsiveness, components, and intermediate animation skills.

Technologies:

- Framer
- Framer Components
- Variants
- Variables
- Animations and Interactions

Tasks:

1. Showcase Design:

- Create a main product showcase section.
- Include the product name, short description, product visual, and call-to-action.
- Maintain proper spacing, typography, alignment, and visual hierarchy.

2. Product Variants:

- Provide at least three selectable product variants.
- Variants may represent colors, models, themes, or other options.
- Selecting a variant should update the displayed product.

3. Components:

- Use reusable Framer components for repeated UI elements.
- Use component properties where appropriate.
- Avoid manually recreating identical elements.

4. Component Variants:

- Use variants to represent different states.

- Create suitable default, hover, selected, or active states.
- State changes should be visually clear.

5. Product Transition:

- Animate the product when switching between variants.
- The transition should feel smooth rather than instantly replacing the content.
- Related text or visual elements may also change with the selected variant.

6. Interactive Details:

- Add interactive points or controls to the product visual.
- Interacting with them should reveal information about a feature of the product.
- Information should appear and disappear smoothly.

7. Scroll Interaction:

- Include at least one scroll-based interaction.
- The interaction may affect the product visual, text, background, or another suitable element.
- The effect should contribute to the presentation instead of being added only for decoration.

8. Responsive Design:

- Create suitable desktop and mobile layouts.
- The showcase should adapt correctly to different screen sizes.
- Interactions should remain usable on touch devices.

9. Animation:

- Use suitable entrance, hover, selection, and transition animations.
- Maintain consistent animation timing.
- Avoid excessive animations that reduce usability.

10. Testing:

- Test all product variants.
- Test hover and click interactions.
- Test scroll interactions.
- Test desktop and mobile layouts.

- Check for overflowing or overlapping elements.
- Verify animations behave correctly.

Optional Enhancements:

- 3D product interaction
- Parallax effects
- Custom cursor interaction
- Advanced scroll effects
- Light/dark product modes
- Custom Code Component
- Animated background
- Product comparison mode

Backend

1. WhistleDrop — Speak Without Being Seen

Description:

Something is wrong inside your organization. People may have information that needs to be reported, but they may not be comfortable revealing their identity.

Build the backend for a confidential reporting system where anyone can submit a report without creating an account or revealing who they are. Moderators should be able to review and manage reports, while the reporter should be able to track the progress of their report without exposing their identity.

The system should maintain privacy throughout the reporting process while still allowing reports to be reviewed and acted upon.

Tasks:

1. Anonymous Reporting:

- Allow users to submit a report without creating an account.
- Each report should contain:
 - Category
 - Description
 - Optional evidence/reference URL
- Suggested categories:
 - Security
 - Harassment
 - Corruption
 - Technical
 - Other

2. Case Tracking:

- Generate a unique case code for every submitted report.
- The case code should be sufficiently difficult to guess.
- Allow reporters to use the case code to check their report status.
- The reporter should not need to reveal their identity.

3. Report Status:

Reports should follow this workflow: SUBMITTED → UNDER_REVIEW → RESOLVED / DISMISSED

- Moderators should be able to update the status.
- Moderators should be able to add a short status update.
- Reporters should be able to view relevant updates using their case code.

4. Moderator Access:

- Provide a secure way for moderators to access reports.
- Moderators should be able to:
 - View reports
 - Filter reports by category/status
 - Update report status
 - Add status updates
- Moderators must not be able to identify who submitted a report.

5. Privacy & Security:

- The API must not expose reporter identity.
- Protect access to reports and moderator functionality.
- Handle invalid and unauthorized requests appropriately.
- Case codes should not be trivially predictable.

6. API Behaviour:

- Use appropriate HTTP methods and status codes.
- Validate incoming data.
- Handle invalid case codes, invalid status changes, missing data, and other incorrect requests.
- Return clear and meaningful responses.

7. Documentation:

The README should contain:

- Setup instructions
- API endpoints and their purpose
- How anonymity is maintained
- Example requests/responses
- Important assumptions and design decisions

Optional Enhancements / Brownie Points

- Moderator/admin dashboard
- Ability to permanently close a case
- Additional privacy protections
- Evidence/file upload
- Search and advanced filtering
- Swagger/OpenAPI documentation
- Automated tests
- Deployment

No frontend/UI is required. The backend can be demonstrated using Postman, Swagger, cURL, or any other suitable method.

2. Offline Sync Conflict — When Devices Disagree

Description:

People increasingly use the same application across multiple devices. A user may make changes on their laptop while offline, make different changes on their phone, and later connect both devices to the server.

What happens when the server receives changes that were made independently from different versions of the same data?

Build a backend that can synchronize changes coming from multiple devices while detecting conflicts instead of silently overwriting existing data.

The system should have clear and consistent rules for deciding what happens when changes can or cannot safely coexist.

Technologies:

Candidates are free to choose any programming language, backend framework, and database suitable for the task. Technology choice will not be evaluated as long as the solution is functional and the candidate can clearly explain their implementation and design decisions.

Tasks:

1. Data Management:

- Allow users to create and modify data through multiple devices.
- The backend should maintain the current state of the data.
- The system should distinguish changes coming from different devices or clients.

2. Change Tracking:

- Each synchronization request should provide enough information for the server to determine the state from which the change was made.
- The backend should identify when a device is working with an outdated version of the data.

3. Conflict Detection:

- Detect when changes from different devices conflict with one another.
- An outdated change must not silently overwrite a newer accepted change.
- Distinguish between changes that can safely coexist and changes that require conflict resolution.

4. Conflict Resolution:

- Define and implement a clear strategy for handling conflicts.
- The strategy should be consistent, preserve data correctly, and produce predictable results.
- Different approaches are acceptable as long as the candidate can explain their approach.

5. Non-Conflicting Changes:

- Changes that do not actually conflict should not unnecessarily be treated as conflicts.
- Preserve valid changes wherever possible.
- Synchronization should not cause unrelated or newer data to be lost.

6. Synchronization Results:

The API should clearly communicate what happened when a change is submitted.

A synchronization attempt may result in:

- Change accepted
- Change rejected
- Changes merged
- Conflict requiring resolution

The exact response structure is up to the candidate, but the client should be able to understand the result.

7. Consistency & Edge Cases:

Handle situations such as:

- Multiple synchronization requests arriving close together
- Changes arriving out of order
- The same synchronization request being sent more than once
- A device attempting to update outdated data
- Two devices modifying the same data independently
- A newer accepted change being accidentally overwritten

8. Validation & API Behaviour:

- Validate synchronization requests and submitted data.
- Handle invalid, stale, conflicting, or unknown requests appropriately.
- Use suitable HTTP methods and status codes.
- Ensure invalid synchronization attempts do not corrupt existing data.

9. Documentation:

The README should explain:

- How synchronization works
- How changes are tracked
- How conflicts are detected
- The chosen conflict-resolution strategy
- Conflicting and non-conflicting scenarios
- Example API requests/responses

Optional Enhancements / Brownie Points

- Field-level conflict detection and merging
- Version/change history
- View previous versions
- Restore an earlier version
- More sophisticated conflict-resolution strategies
- Swagger/OpenAPI documentation
- Automated tests
- Deployment

No frontend/UI is required. The backend can be demonstrated using Postman, Swagger, cURL, or any other suitable method.

App Development

1. Digital Archive

Description :

Build a mobile application that allows users to import files into a personal archive, organize them using metadata and tags, and search/filter the archive.

The application must maintain a persistent relationship between the archive records and the actual files.

Technologies:

- Any mobile or cross-platform framework (SwiftUI, Flutter, React Native, KMP, or Web)
- Local data storage of your choice
- Background execution capabilities supported by your platform

Tasks:

1. File Import:

- Allow users to select and import files from the device.
- Support at least 3 file types such as images, PDFs, and documents.
- Handle multiple-file imports.
- Store the required metadata when a file is imported.

2. File Metadata:

Each archived file should contain:

- File name
- File type
- File size
- Import date
- Last known modification date
- Tags/categories
- File reference/location
- Availability status

3. Organization:

- Create/delete tags or categories.

- Assign/remove tags from files.
- Rename an archive entry.
- Remove an entry from the archive.
- Clearly define whether removing an entry also removes the physical file.

4. Search & Filtering:

- Search by filename and tags/categories.
- Provide at least one additional filter such as file type, size, date, or availability.
- Search/filter must work using persisted archive data.

5. File Availability:

Handle cases where a file:

- Has been deleted externally.
- Has been moved.
- Is no longer accessible.
- Cannot be opened.

The app must not crash, and the archive metadata should remain available.

6. Persistence:

- Archive data must survive application restarts.
- Do not rebuild the archive entirely from the device on every launch.
- Use persistent local storage/database.

7. Import Consistency:

Handle failures such as:

- Import cancellation
- File-copy failure
- Database failure
- Application termination during import
- Partial success during multi-file import

The database must not contain invalid records for files that were never successfully imported.

8. Async Operations:

File copying, metadata extraction, and other large file operations must not block the UI.

Expected

The application should demonstrate:

- Mobile UI
- Local database/persistence
- File-system interaction
- Search/filtering
- Async file operations
- Missing-file handling
- Separation of UI, application logic, and storage

Optional Enhancements

- Duplicate detection using file content or other meaningful properties
- Archive integrity scanner that detects missing/unreadable files
- Automated tests

Submission

- Source code
- Working APK
- README containing architecture, storage strategy, and important edge cases handled

2. Resumable File Transfer

Description :

Build a mobile application that manages large file uploads/downloads while handling interruptions, failures, application restarts, and unreliable network conditions.

A local/mock server may be used.

Technologies:

- Any mobile or cross-platform framework (SwiftUI, Flutter, React Native, KMP, or Web)
- Local data storage of your choice
- Background execution capabilities supported by your platform

Requirements

1. Upload & Download:

- Support both file upload and download.
- Transfers should work with sufficiently large files to demonstrate interruption and recovery.

2. Transfer Queue:

Maintain multiple transfers with:

- File name
- File size
- Progress
- Transfer type
- Current state
- Error information where applicable

3. Transfer States:

At minimum support states equivalent to: QUEUED → TRANSFERRING → COMPLETED

And: TRANSFERRING → RETRYING / FAILED / PAUSED

State must be persisted and must not exist only in the UI.

4. Pause & Resume:

- A transfer can be paused.

- Resuming must continue from previously completed work rather than restarting from zero.

5. Persistent Progress:

Example: 40% transferred → App terminated → App reopened → Transfer restored at 40%

The application must store enough information to determine what work remains.

6. Partial Transfers:

Large files should be divided into manageable portions/chunks or use another mechanism that supports partial progress.

Example: Chunk 1 ✓ | Chunk 2 ✓ | Chunk 3 X | Chunk 4 —

Recovery should retry only the required work.

7. Failure & Retry:

Handle:

- Network disconnection
- Timeouts
- Temporary server errors
- Failed chunks/requests
- Application termination

Retry must not create an infinite loop or destroy valid progress.

8. Duplicate/Lost Requests:

Handle situations such as: Client sends chunk → Server receives it → Response is lost → Client times out

The system must determine whether the chunk was already processed rather than blindly corrupting or duplicating the transfer.

9. Data Integrity:

A transfer should only be marked COMPLETED when the resulting file is verified to be complete and valid.

10. Concurrency:

- Support multiple queued transfers.

- The candidate must decide whether transfers are sequential, limited-concurrent, or fully concurrent and implement the chosen approach correctly.

11. Lifecycle:

Handle:

- Background/foreground transitions
- Application termination
- Application restart

The application must not falsely mark an interrupted transfer as completed.

12. Cancellation:

- Allow active/queued transfers to be cancelled.
- A cancelled transfer must not later resume automatically as a paused transfer.

Expected

The application should demonstrate:

- Mobile UI
- Local database/persistence
- File-system operations
- Networking
- Async/background processing
- Transfer queue
- Pause/resume
- Retry and recovery
- Lifecycle handling
- Data integrity

Optional Enhancements

- Transfer speed and ETA
- Configurable concurrency
- Transfer history/logs
- Network-condition simulator
- Recovery from corrupted partial data without restarting the entire transfer
- Automated tests

Submission

- Source code
- Working APK

README containing architecture, transfer protocol, persistence strategy, retry/recovery logic, and important edge cases handled